RECRUITMENT & ASSESSMENT PLATFORM

Optimized database architecture

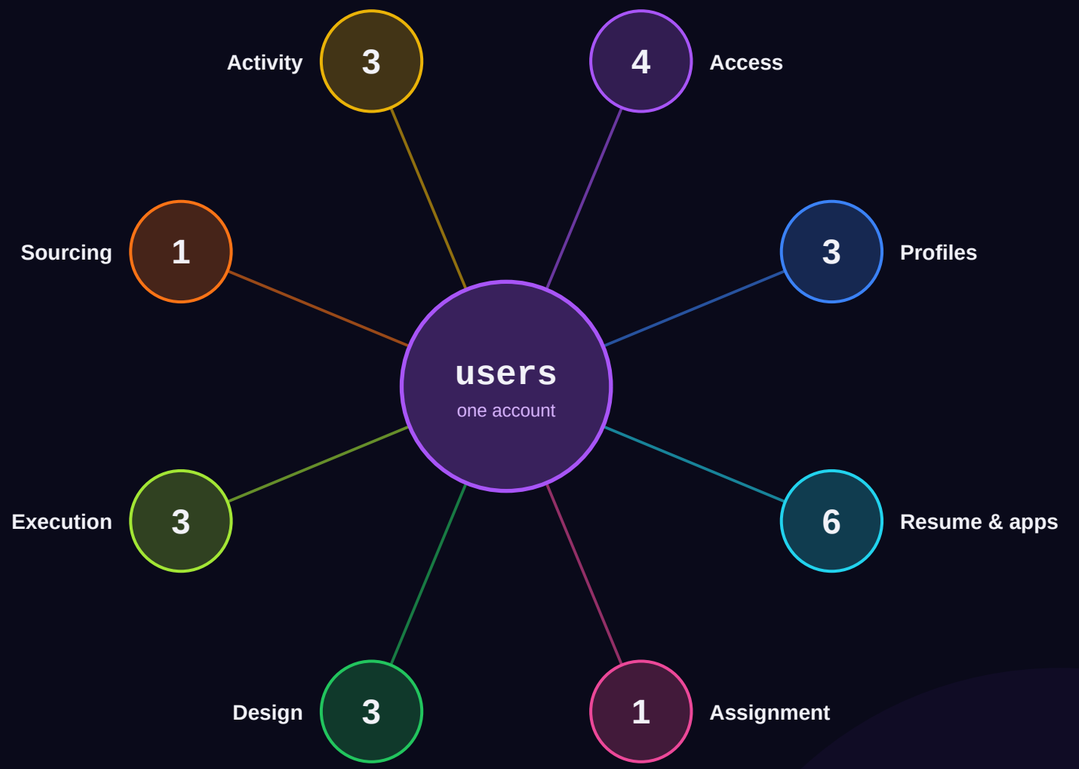
Tables, columns and how the data connects, after the squeeze

24
tables
was 32

142
columns
was 215

33
declared FKs
was 48

8
table modules
unchanged



Source: platform database architecture document, optimized per the schema review · PostgreSQL, user UUID + BIGINT keys

Generated 2 November 2026 · 24 core tables + 1 restricted company search view

At a glance

What is in the optimized database and how tightly it is connected

OVERVIEW

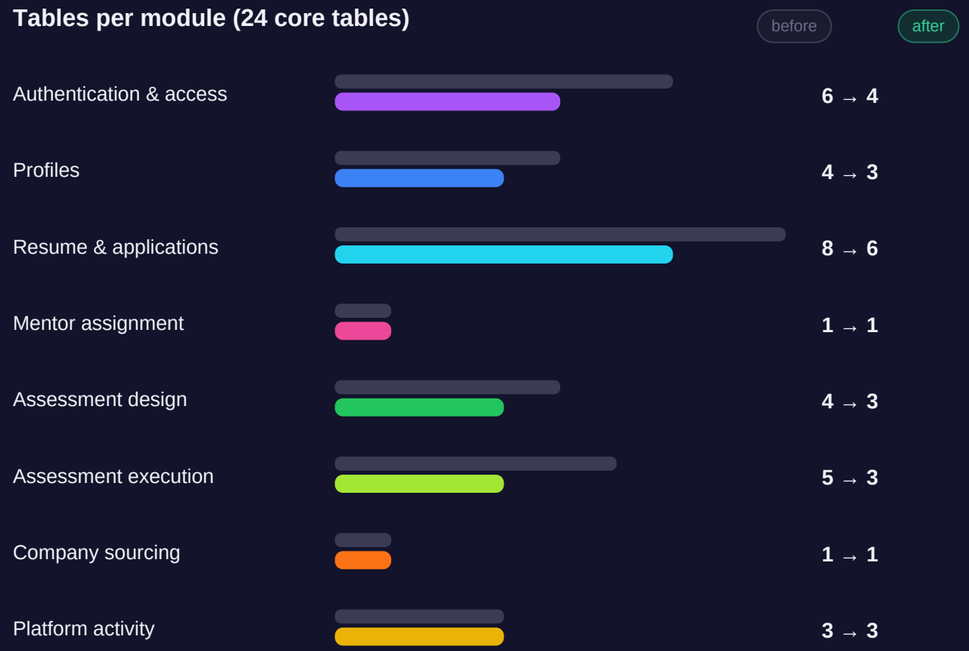
24
tables in the schema
was 32 · -8 (25%)
+1 restricted view: company_candidate_search

142
columns
was 215 · -73 (34%)
avg. 5.9 per table (was 6.7)

33
declared foreign keys
was 48 · -15 (31%)
14 point at users.id

5
platform roles
now a CHECK, no roles table
super_admin · admin · mentor · candidate · company

Tables per module (24 core tables)



users is the hub

14 of 33 foreign keys point at users.id. Candidates, mentors and company staff still sign in through one account; a person can be mentor and candidate without a second login.

Assignment is the spine

Rounds hang off mentor_assignments, which already holds the candidate application and the mentor. Attempts, answers and decisions resolve the candidate through round → assignment → application, so no id is duplicated.

Consent before contact

Companies search a restricted view with no name, email, phone or resume link. introduction_requests records the candidate's consent before any contact detail is released.

Scores keep provenance

candidate_answers stores auto, AI and mentor scores in separate columns; final_score is generated as COALESCE(mentor, AI, auto), so automation can never overwrite a mentor.

Table merges

Eight tables removed: a table merges only when it is strictly 1:1 with its parent

OPTIMIZATION

`roles` `permissions`	→	**Dropped; role and permission_key become text columns with a CHECK**	The platform has exactly five fixed roles, and permission keys live in code
`candidate_profiles` `mentor_profiles`	→	**profiles (PK = user_id)**	Both are 1:1 with users and share title, experience and bio
`candidate_skills` `mentor_skills`	→	**user_skills**	Same shape; proficiency is simply nullable for mentors
`resume_analysis`	→	**Columns on resumes**	It is one parse result per resume
`assessments`	→	**assessment_rounds.mentor_assignment_id**	An assessment already hangs 1:1 off an assignment, so rounds can point there directly
`assessment_results`	→	**Columns on assessment_attempts**	1:1 with an attempt (it had UNIQUE attempt_id)
`round_decisions`	→	**Decision columns on assessment_rounds**	A round belongs to exactly one candidate, so it has one current decision

32 tables → 24 tables — Seven merge groups remove eight tables net; the restricted view company_candidate_search is kept and is not counted.

Columns removed

A column goes if it is derivable, duplicated, or read by no workflow

OPTIMIZATION

Derivable from other data

profile_completion
computed from the other profile fields

objective_score, descriptive_score, practical_score, percentage
a SUM and GROUP BY over candidate_answers

file_type
comes from the file extension

approval_status, approved_by, approved_at (mentor)
already exist as applications.status, reviewed_by and reviewed_at; "suspended" becomes user_roles.is_active = false

Duplicated elsewhere

attempts.candidate_id; round_decisions.candidate_id and mentor_id
round → assignment → application leads to both

attempts.total_score
repeated assessment_results.total_score

evaluation_status
folded into attempts.status: in_progress, submitted, evaluated, expired

level.sequence_order
duplicated level_number

evaluation_criteria
folded into answer_key

answer_text, selected_options
merged into one answer JSONB

final_score
now generated: COALESCE(mentor, AI, auto)

Surrogate id on junction tables
replaced by composite primary keys

Unused by any flow

updated_at
removed everywhere

Descriptions
on levels, rounds, job roles; notification title

skills.category

company_members.member_role

companies: company_size, location, description

analysed_at; per-answer evaluated_at

created_at
on user_roles and assessment_rounds

Resume parse split
skills, employment and education fold into parsed_data JSONB

215 → 142 columns **-73 columns (34%)** average per table 6.7 → 5.9 declared FKs 48 → 33

End-to-end flow

The five role flows the schema is built around, and where each lands

OVERVIEW

1 Super Admin

WHAT IT DOES

- Create admin accounts
- Set access control (permissions)
- Manage platform settings and policies

WHERE IT LANDS

`users, user_roles, user_permissions, audit_logs`

2 Admin

WHAT IT DOES

- Receive applications (resume + apply as candidate or mentor)
- Analyse the resume, check experience
- 10+ years: mentor application allowed; otherwise candidate
- Review, approve or reject
- Assign an approved mentor

WHERE IT LANDS

`applications, resumes (parse), user_roles, mentor_assignments`

3 Mentor

WHAT IT DOES

- View relevant candidates
- Build rounds, levels, questions and answer keys
- Monitor tab switches, time, events
- Decide: proceed, reject, hold, reschedule
- Rank candidates, track engagement

WHERE IT LANDS

`assessment_rounds → levels → questions, round decision columns`

4 Candidate

WHAT IT DOES

- Complete profile, upload resume
- Apply for roles (candidate or mentor)
- Attempt assessments
- View results and feedback
- Track profile, applications, status

WHERE IT LANDS

`profiles, resumes, applications, assessment_attempts`

5 Company

WHAT IT DOES

- Search and filter by role, skills, experience
- View anonymous profiles: scores, skills, experience
- Request an introduction, no contact details

WHERE IT LANDS

`company_candidate_search (view), introduction_requests`

PLATFORM OUTCOMES

Fair & transparent assessment | Candidate privacy protected | Qualified talent for companies | Detailed analytics & insights

A seamless, secure and intelligent recruitment ecosystem: from application to assessment to hiring, all in one platform.

Every flow still works

What each role writes to after optimization, and what changed underneath

WORKFLOW CHECK

ROLE	WRITES TO	WHAT CHANGED UNDERNEATH	STATUS
Super Admin	`users, user_roles, user_permissions, audit_logs`	The roles, permissions and role_permissions catalogue tables are replaced by CHECK-constrained text keys.	Unchanged
Admin	`applications, resumes, user_roles, mentor_assignments`	Resume parse columns sit on resumes. Mentor approval uses applications.status / reviewed_by / reviewed_at; approving inserts an active mentor row in user_roles.	Unchanged
Mentor	`assessment_rounds, assessment_levels, assessment_questions, round decision columns`	The assessments row is gone: rounds point at the assignment. round_decisions and assessment_results are columns on the round and the attempt.	Works
Candidate	`profiles, resumes, user_skills, applications, assessment_attempts, candidate_answers`	One profiles table. The attempt carries the result summary. The candidate resolves through round → assignment → application.	Unchanged
Company	`companies, company_members, introduction_requests, company_candidate_search`	Consent gate intact: the view reads profiles.is_discoverable plus an approved application and never exposes contact data.	Unchanged

Index assessment_rounds.mentor_assignment_id: candidate lookups now go through round → assignment → application instead of a stored candidate_id.

Domain map

Eight modules of tables, all connected back to the users table

OVERVIEW

Authentication & access

4 tables (was 6)

users	8
user_roles	4
role_permissions	2
user_permissions	4

Profiles

3 tables (was 4)

profiles	9
companies	5
company_members	2

Resume & applications

6 tables (was 8)

resumes	9
skills	2
user_skills	4
job_roles	3
role_skills	2
applications	10

Mentor assignment

1 table (was 1)

mentor_assignments	7

Assessment design

3 tables (was 4)

assessment_rounds	11
assessment_levels	4
assessment_questions	8

Assessment execution

3 tables (was 5)

assessment_attempts	10
candidate_answers	7
assessment_events	5

Company sourcing

1 table (was 1)

introduction_requests	8

Platform activity

3 tables (was 3)

engagement_events	5
notifications	6
audit_logs	7

users

14 FK children · one login, many roles

Every profile, application, resume, membership, notification and audit entry traces back to one account.

How one candidate moves through the data

OVERVIEW

From sign-up to a consented introduction, and the tables written at each step

1 Sign up

`users, user_roles`

A person registers once. Rows in user_roles decide which dashboards open; a Super Admin grants admin capabilities through user_permissions.

2 Profile & resume

`profiles, resumes, user_skills`

The file goes to private object storage; resumes holds the reference plus the experience estimate and parsed skills.

3 Apply & review

`applications, mentor_assignments`

One applications table serves candidates and mentors. An admin reviews, approves or rejects, then assigns an approved mentor.

4 Assess

`assessment_rounds, attempts, answers, events`

The mentor builds rounds, levels and questions. Each attempt stores answers, integrity events and scores; the decision sits on the round.

5 Get discovered

`company_candidate_search, introduction_requests, notifications`

Approved, discoverable candidates appear anonymously in a restricted view. Contact is released only after the candidate accepts.

Decisions stay separate from scores

Automated scoring is an initial aid. candidate_answers keeps auto_score, ai_score and mentor_score as separate columns, and final_score is generated as COALESCE(mentor_score, ai_score, auto_score). The round's decision (proceed, reject, hold, reschedule) is written by the mentor in its own columns on assessment_rounds.

Mentor eligibility follows the same rule: resume analysis gives a rough experience estimate (over 10 years) and an admin confirms it by human review before the mentor application is approved and the mentor role activated.

Consent gate for company access

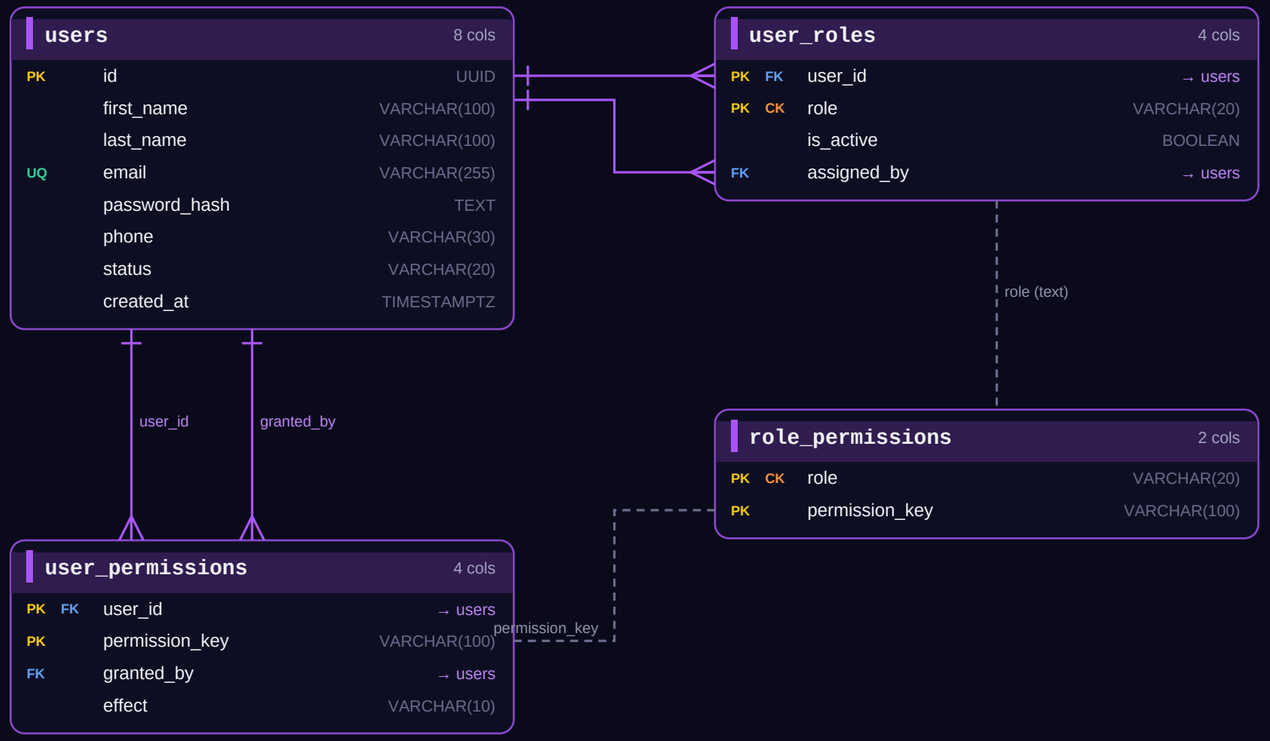
1. Company searches the restricted view
2. Requests an introduction
3. Candidate accepts or rejects
4. Contact details released

The view returns an opaque candidate reference, title, broad location, skills, experience band and a permitted assessment summary. Name, email, phone, exact address, resume URL, private answers and mentor-only notes never leave it. One active request per company and candidate pair.

Access control

One account table; roles and permissions are plain keys on junction tables

ER · ACCESS



How access resolves

Effective permissions start from role_permissions for each active role in user_roles; user_permissions then allows or denies an individual capability. A Super Admin grants admin capabilities through user_permissions.granted_by.

Rules on this page

PRIMARY KEY (user_id, role) and PRIMARY KEY (role, permission_key) replace surrogate ids. Validate the active dashboard role against user_roles. Store password_hash only, never plain text, and check users.status at login.

No roles or permissions tables

role is VARCHAR(20) with CHECK (role IN ('super_admin','admin','mentor','candidate','company')). permission_key values live in application code and are validated on write. Adding a sixth role needs a migration.

Five platform roles

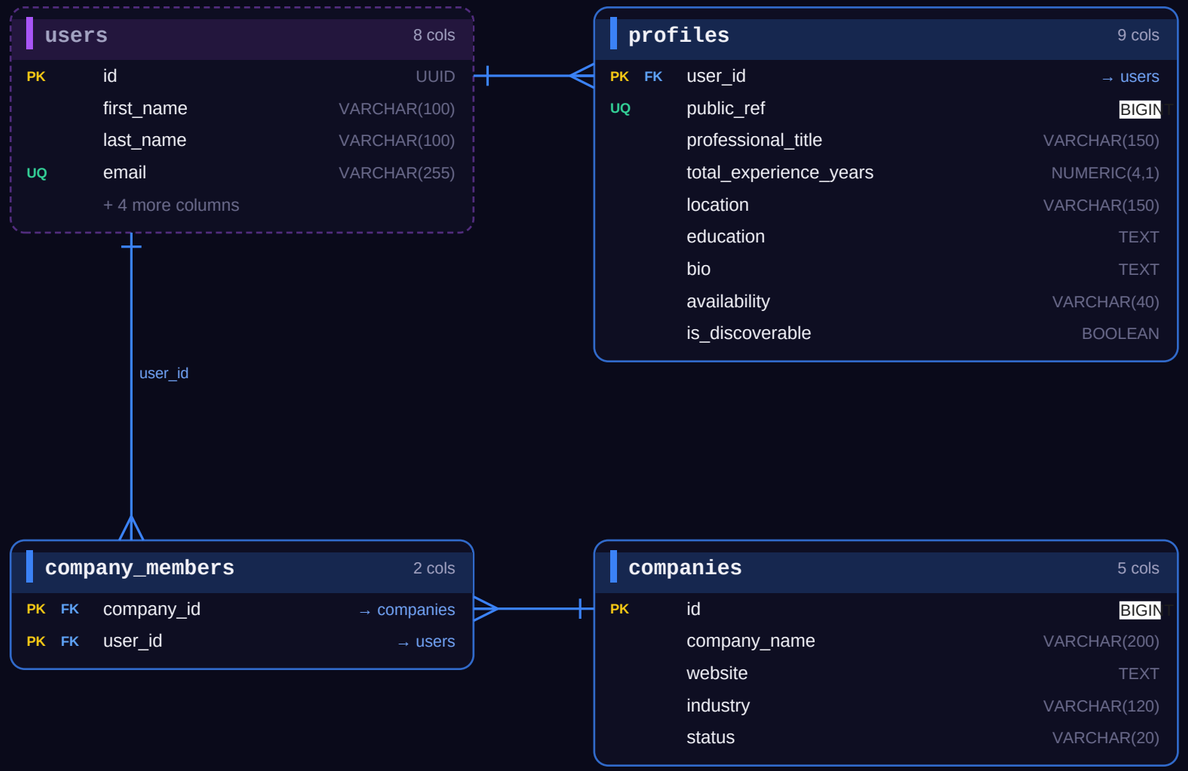
`super_admin` `admin` `mentor` `candidate` `company`

PK primary key FK declared foreign key UQ unique CK check GEN generated column · crow's foot = many (child), bar = one (parent), dashed = logical link · '→ table' = parent not drawn on this page

Profiles and company membership

One shared profile per person; companies are reached through company_members

ER · PROFILES



One user, one profile

profiles is keyed by user_id, so a person who is both mentor and candidate shares one title, experience and bio. companies has no user_id: employees join through company_members, so several people can share one company.

Consent lives on the profile

profiles.is_discoverable records the candidate's visibility consent. Only approved, discoverable candidates are exposed to companies, and only through the restricted search view. profiles.public_ref is a random UUID, so the view never exposes user_id.

Mentor approval is an admin decision

The mentor_profiles approval columns are gone. An admin approves a mentor application (applications.status, reviewed_by, reviewed_at) and an active mentor row is inserted in user_roles. Suspending a mentor sets user_roles.is_active = false.

Derived, not stored

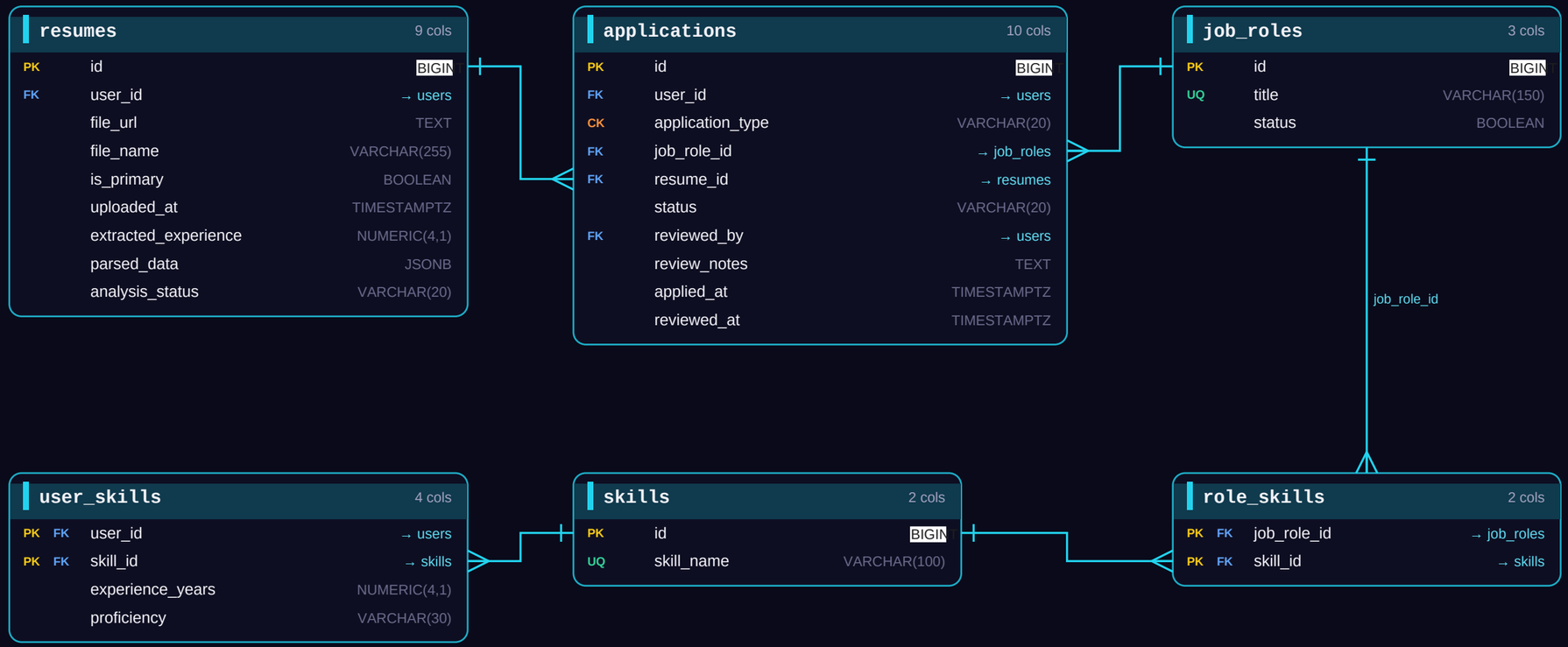
profile_completion is computed from the filled fields when needed, so it can never drift from the data.

PK primary key FK declared foreign key UQ unique CK check GEN generated column · crow's foot = many (child), bar = one (parent), dashed = logical link · '→ table' = parent not drawn on this page

Resumes, skills and applications

Private resume files with their parse result, one skill catalogue and one application record

ER · RESUME & APPS



Private resume storage

resumes keeps metadata and a private object reference, never the file as a database blob. Issue short-lived, authorised links.

Parse result on the row

extracted_experience is an estimate; parsed_data holds skills, employment and education. analysis_status can be needs_review so a human stays in the loop.

Two meanings of role

job_roles are professional tracks such as React Developer. Platform roles live in user_roles.role.

One application table

application_type (candidate or mentor) shares one record shape. Internal review_notes stay hidden from applicants.

Matching skills

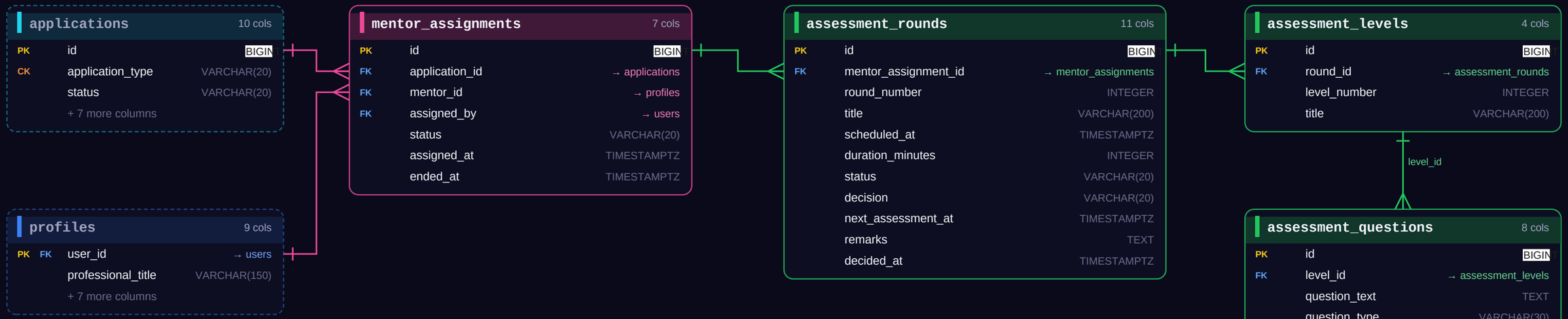
user_skills serves candidates and mentors alike (proficiency is NULL for mentors) and both point at the shared skills catalogue, while role_skills links the same catalogue to job_roles. Matching a candidate to a mentor is a join across those three tables.

PK primary key FK declared foreign key UQ unique CK check GEN generated column · crow's foot = many (child), bar = one (parent), dashed = logical link · '→ table' = parent not drawn on this page

Mentor assignment and assessment design

An approved application is assigned to a mentor, who builds rounds, levels and questions

ER · ASSESSMENT DESIGN



Assignment rules

Validate that the application is an approved candidate application and the mentor holds an active mentor role. Keep assignment history when reassigning (active, completed, reassigned, cancelled).

Rounds point at the assignment

mentor_assignments already identifies the candidate application and the mentor, so rounds need one reference instead of two that could disagree. The separate assessments table is gone: one plan per assignment.

Unlimited rounds and levels

UNIQUE (mentor_assignment_id, round_number) and UNIQUE (round_id, level_number). Types: mcq, multiple_select, true_false, short_answer, descriptive, coding, practical. answer_key holds the expected answer and rubric.

Snapshot before attempts

Version or snapshot published questions so later edits never change an in-progress or historical assessment. Decide the strategy before production.

PK primary key FK declared foreign key UQ unique CK check GEN generated column · crow's foot = many (child), bar = one (parent), dashed = logical link · '→ table' = parent not drawn on this page

Attempts, monitoring and results

A candidate starts a round; answers, events and scores stay apart from the mentor's decision

ER · EXECUTION



Who is the candidate?

assessment_attempts no longer stores candidate_id. The round points at its mentor assignment, the assignment at the application, and the application at the candidate user.

Score provenance

candidate_answers separates auto_score, ai_score and mentor_score; final_score is generated as COALESCE(mentor_score, ai_score, auto_score), so a mentor score is never silently overwritten by automation.

Events are signals, not proof

tab_hidden, window_blur, fullscreen_exit, network_disconnected and similar events are imperfect. Disclose monitoring to candidates and apply retention limits.

Result and decision are separate fields

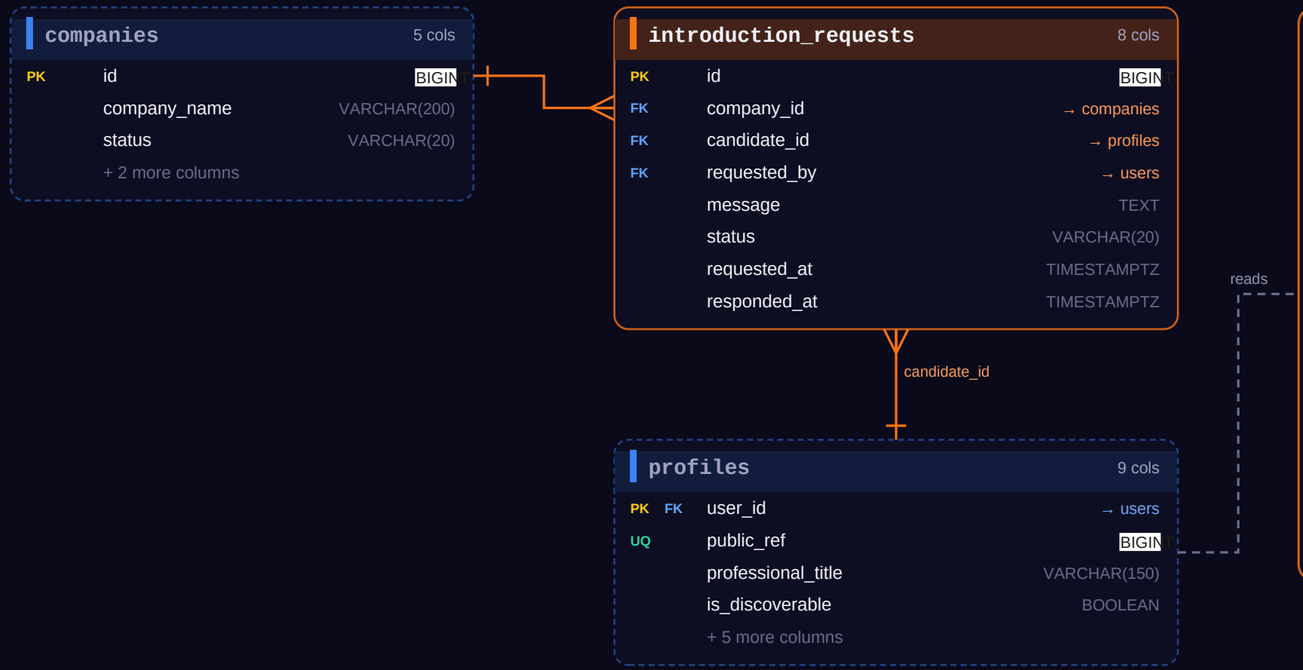
Scores and feedback sit on the attempt. The mentor's call (proceed, reject, hold, reschedule) sits on the round: one current decision, with history kept in audit_logs.

PK primary key FK declared foreign key UQ unique CK check GEN generated column · crow's foot = many (child), bar = one (parent), dashed = logical link · '→ table' = parent not drawn on this page

Company sourcing

Companies search a restricted view and can only request a consented introduction

ER · SOURCING



company_candidate_search

RESTRICTED VIEW

A SQL view, not a duplicate candidate table. Approved and discoverable candidates only.

EXPOSES

- opaque candidate reference (public_ref)
- professional title
- broad location
- skills
- experience band
- permitted assessment summary
- availability

NEVER EXPOSES

- name
- email
- phone
- exact address
- original resume URL
- private answers
- mentor-only notes

reads only approved + discoverable profiles

INTRODUCTION LIFECYCLE (introduction_requests.status)

pending

A company member (requested_by) sends a message about one candidate. One active request per company and candidate pair.

accepted

The candidate agrees. Record the consent time in responded_at; contact details may now be released.

rejected

The candidate declines. No contact detail is ever exposed, and the company cannot re-send while an active request exists.

withdrawn

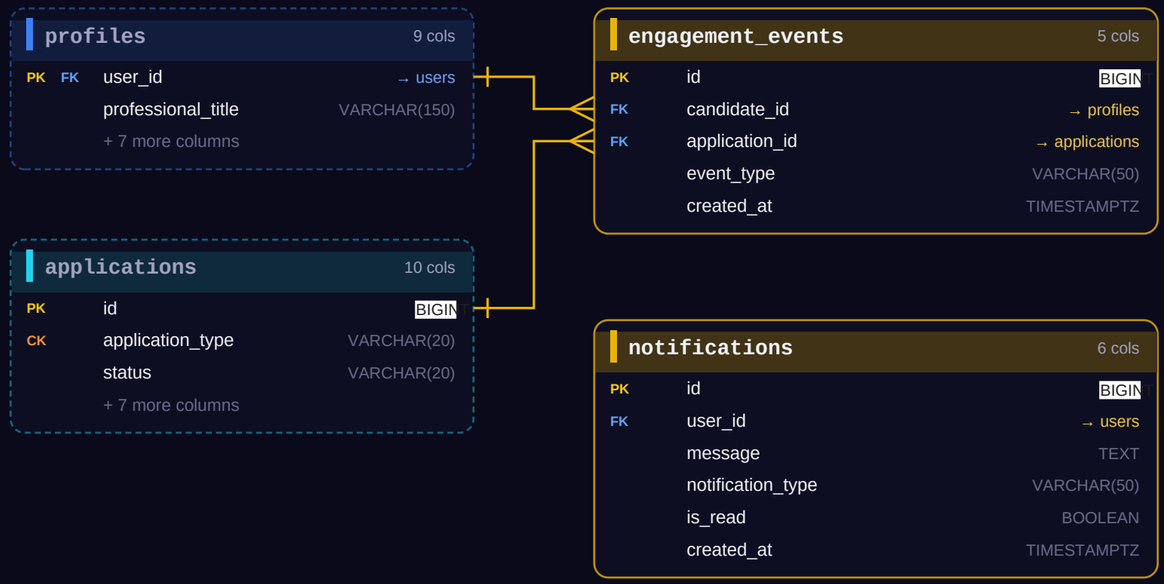
The company cancels before a response. Use a transaction for the consent step so release and record never drift apart.

PK primary key FK declared foreign key UQ unique CK check GEN generated column · crow's foot = many (child), bar = one (parent), dashed = logical link · '→ table' = parent not drawn on this page

Platform activity

Engagement signals, in-app notifications and the append-only audit trail

ER · ACTIVITY





Event and notification types

ENGAGEMENT

```
schedule_viewed
instructions_viewed
assessment_started
assessment_submitted
profile_completed
```

NOTIFICATIONS

```
application
mentor
assessment
result
introduction
```

Engagement events

Log meaningful events between rounds, not every page refresh. Disclose the tracking. Engagement is not a measure of enthusiasm or ability.

Notifications

notification_type tells the app how to route the message. Index user_id, is_read and created_at for the unread badge. The title column was dropped; message carries the text.

Audit trail

Append-only record of sensitive platform and admin actions, now also holding earlier round decisions. Restrict update and delete. Never place passwords, tokens, resume contents or private answers in details.

PK primary key FK declared foreign key UQ unique CK check GEN generated column · crow's foot = many (child), bar = one (parent), dashed = logical link · '→ table' = parent not drawn on this page

Table inventory

Every table by module, with a short purpose and column count

REFERENCE

Authentication & access — 4

users	Central login account for everyone	8
user_roles	Accounts to roles; role is a CHECK-constrained text column	4
role_permissions	Default permissions per role (permission_key is text)	2
user_permissions	Per-user grants / overrides	4

Profiles — 3

profiles	Candidate and mentor profile, one row per user	9
companies	Company account details	5
company_members	Employees of a company	2

Resume & applications — 6

resumes	Private file metadata plus parse result	9
skills	Central skill catalogue	2
user_skills	Skills of candidates and mentors	4
job_roles	Job tracks (not login roles)	3
role_skills	Skills per job role	2
applications	Candidate and mentor applications	10

Mentor assignment — 1

mentor_assignments	Application to responsible mentor	7

Assessment design — 3

assessment_rounds	Rounds of an assignment, with the mentor decision	11
assessment_levels	Ordered levels in a round	4
assessment_questions	Questions, answer keys and marks	8

Assessment execution — 3

assessment_attempts	One row per attempt, with result summary	10
candidate_answers	Every answer in an attempt	7
assessment_events	Integrity event timeline	5

Company sourcing — 1

introduction_requests	Company asks to meet a candidate	8

Platform activity — 3

engagement_events	Meaningful candidate activity	5
notifications	In-app notifications	6
audit_logs	Append-only sensitive actions	7

company_candidate_search is a restricted SQL view and is not counted as a table. Column counts come from the optimized schema.

Column reference (1/3)

Full column lists: users to resumes

COLUMNS

users — 8 cols

PK	id	UUID
	first_name	VARCHAR(100)
	last_name	VARCHAR(100)
UQ	email	VARCHAR(255)
	password_hash	TEXT
	phone	VARCHAR(30)
	status	VARCHAR(20)
	created_at	TIMESTAMPTZ

user_roles — 4 cols

PK FK	user_id	→ users
PK CK	role	VARCHAR(20)
	is_active	BOOLEAN
FK	assigned_by	→ users

role_permissions — 2 cols

PK CK	role	VARCHAR(20)
PK	permission_key	VARCHAR(100)

user_permissions — 4 cols

PK FK	user_id	→ users
PK	permission_key	VARCHAR(100)
FK	granted_by	→ users
	effect	VARCHAR(10)

profiles — 9 cols

PK FK	user_id	→ users
UQ	public_ref	BIGINT
	professional_title	VARCHAR(150)
	total_experience_years	NUMERIC(4,1)
	location	VARCHAR(150)
	education	TEXT
	bio	TEXT
	availability	VARCHAR(40)
	is_discoverable	BOOLEAN

companies — 5 cols

PK	id	BIGINT
	company_name	VARCHAR(200)
	website	TEXT
	industry	VARCHAR(120)
	status	VARCHAR(20)

company_members — 2 cols

PK FK	company_id	→ companies
PK FK	user_id	→ users

resumes — 9 cols

PK	id	BIGINT
FK	user_id	→ users
	file_url	TEXT
	file_name	VARCHAR(255)
	is_primary	BOOLEAN
	uploaded_at	TIMESTAMPTZ
	extracted_experience	NUMERIC(4,1)
	parsed_data	JSONB
	analysis_status	VARCHAR(20)

Types are PostgreSQL types. UUID primary keys, created_at where a record needs a creation time. Composite primary keys on pure junction tables.

Column reference (2/3)

Full column lists: skills to assessment_questions

COLUMNS

skills — 2 cols

PK	id	BIGINT
UQ	skill_name	VARCHAR(100)

user_skills — 4 cols

PK	FK	user_id	→ users
PK	FK	skill_id	→ skills
		experience_years	NUMERIC(4,1)
		proficiency	VARCHAR(30)

job_roles — 3 cols

PK	id	BIGINT
UQ	title	VARCHAR(150)
	status	BOOLEAN

role_skills — 2 cols

PK	FK	job_role_id	→ job_roles
PK	FK	skill_id	→ skills

applications — 10 cols

PK	id	BIGINT
FK	user_id	→ users
CK	application_type	VARCHAR(20)
FK	job_role_id	→ job_roles
FK	resume_id	→ resumes
	status	VARCHAR(20)
FK	reviewed_by	→ users
	review_notes	TEXT
	applied_at	TIMESTAMPTZ
	reviewed_at	TIMESTAMPTZ

mentor_assignments — 7 cols

PK	id	BIGINT
FK	application_id	→ applications
FK	mentor_id	→ profiles
FK	assigned_by	→ users
	status	VARCHAR(20)
	assigned_at	TIMESTAMPTZ
	ended_at	TIMESTAMPTZ

assessment_rounds — 11 cols

PK	id	BIGINT
FK	mentor_assignment_id	→ mentor_assignments
	round_number	INTEGER
	title	VARCHAR(200)
	scheduled_at	TIMESTAMPTZ
	duration_minutes	INTEGER
	status	VARCHAR(20)
	decision	VARCHAR(20)
	next_assessment_at	TIMESTAMPTZ
	remarks	TEXT
	decided_at	TIMESTAMPTZ

assessment_levels — 4 cols

PK	id	BIGINT
FK	round_id	→ assessment_rounds
	level_number	INTEGER
	title	VARCHAR(200)

assessment_questions — 8 cols

PK	id	BIGINT
FK	level_id	→ assessment_levels
	question_text	TEXT
	question_type	VARCHAR(30)
	options	JSONB
	answer_key	TEXT
	marks	NUMERIC(7,2)
	sequence_order	INTEGER

Types are PostgreSQL types. UUID primary keys, created_at where a record needs a creation time. Composite primary keys on pure junction tables.

Column reference (3/3)

Full column lists: assessment_attempts to audit_logs

COLUMNS

assessment_attempts — 10 cols

PK	id	BIGIN
FK	round_id	→ assessment_rounds
	attempt_number	INTEGER
	started_at	TIMESTAMPTZ
	submitted_at	TIMESTAMPTZ
	status	VARCHAR(20)
	total_score	NUMERIC(7,2)
	ai_feedback	TEXT
	mentor_feedback	TEXT
	evaluated_at	TIMESTAMPTZ

candidate_answers — 7 cols + 1 generated

PK FK	attempt_id	→ assessment_attempts
PK FK	question_id	→ assessment_questions
	answer	JSONB
	auto_score	NUMERIC(7,2)
	ai_score	NUMERIC(7,2)
	mentor_score	NUMERIC(7,2)
	feedback	TEXT
GEN	final_score	NUMERIC(7,2)

assessment_events — 5 cols

PK	id	BIGIN
FK	attempt_id	→ assessment_attempts
	event_type	VARCHAR(50)
	event_timestamp	TIMESTAMPTZ
	event_details	JSONB

introduction_requests — 8 cols

PK	id	BIGIN
FK	company_id	→ companies
FK	candidate_id	→ profiles
FK	requested_by	→ users
	message	TEXT
	status	VARCHAR(20)
	requested_at	TIMESTAMPTZ
	responded_at	TIMESTAMPTZ

engagement_events — 5 cols

PK	id	BIGIN
FK	candidate_id	→ profiles
FK	application_id	→ applications
	event_type	VARCHAR(50)
	created_at	TIMESTAMPTZ

notifications — 6 cols

PK	id	BIGIN
FK	user_id	→ users
	message	TEXT
	notification_type	VARCHAR(50)
	is_read	BOOLEAN
	created_at	TIMESTAMPTZ

audit_logs — 7 cols

PK	id	BIGIN
FK	actor_id	→ users
	action	VARCHAR(100)
	entity_type	VARCHAR(80)
	entity_id	BIGIN
	details	JSONB
	created_at	TIMESTAMPTZ

Types are PostgreSQL types. UUID primary keys, created_at where a record needs a creation time. Composite primary keys on pure junction tables.

Foreign key catalogue

All 33 declared constraints, child.column → parent.key

RELATIONS

- user_roles.user_id → users.id
- user_roles.assigned_by → users.id
- user_permissions.user_id → users.id
- user_permissions.granted_by → users.id
- profiles.user_id → users.id
- company_members.company_id → companies.id
- company_members.user_id → users.id
- resumes.user_id → users.id
- user_skills.user_id → users.id
- user_skills.skill_id → skills.id
- role_skills.job_role_id → job_roles.id
- role_skills.skill_id → skills.id
- applications.user_id → users.id
- applications.job_role_id → job_roles.id
- applications.resume_id → resumes.id
- applications.reviewed_by → users.id
- mentor_assignments.application_id → applications.id
- mentor_assignments.mentor_id → profiles.user_id
- mentor_assignments.assigned_by → users.id
- assessment_rounds.mentor_assignment_id → mentor_assignments.id
- assessment_levels.round_id → assessment_rounds.id
- assessment_questions.level_id → assessment_levels.id
- assessment_attempts.round_id → assessment_rounds.id
- candidate_answers.attempt_id → assessment_attempts.id
- candidate_answers.question_id → assessment_questions.id
- assessment_events.attempt_id → assessment_attempts.id
- introduction_requests.company_id → companies.id
- introduction_requests.candidate_id → profiles.user_id
- introduction_requests.requested_by → users.id
- engagement_events.candidate_id → profiles.user_id
- engagement_events.application_id → applications.id
- notifications.user_id → users.id
- audit_logs.actor_id → users.id

Bar colour shows the child table's module. Every FK is enforced by the database; add ON DELETE rules per workflow (for example, keep assignment history on reassign). 14 of 33 point at users.id.

Constraints and indexes

Recommended database rules, before any application logic

REFERENCE

Recommended constraints

Accounts	UNIQUE (users.email); password_hash only; validate account status at login.
Roles	PRIMARY KEY (user_roles.user_id, role); CHECK role IN ('super_admin','admin','mentor','candidate','company'). A user may hold several roles.
Profiles	profiles.user_id is both PRIMARY KEY and FOREIGN KEY; UNIQUE (public_ref).
Company membership	PRIMARY KEY (company_members.company_id, user_id).
Skills	UNIQUE (skill_name); PRIMARY KEY (user_skills.user_id, skill_id); PRIMARY KEY (role_skills.job_role_id, skill_id).
Applications	CHECK application_type IN ('candidate','mentor'); job_role_id references job_roles.
Assessments	UNIQUE (mentor_assignment_id, round_number); UNIQUE (round_id, level_number).
Attempts	UNIQUE (round_id, attempt_number). The candidate is resolved through the round's assignment.
Answers	PRIMARY KEY (attempt_id, question_id); final_score GENERATED ALWAYS AS (COALESCE(mentor_score, ai_score, auto_score)) STORED.
Results	None needed: the result summary is columns on assessment_attempts, one row per attempt.
Decisions	One current decision per round (columns on assessment_rounds); earlier decisions are written to audit_logs.
Introductions	Partial UNIQUE (company_id, candidate_id) WHERE status IN ('pending','accepted'), so no duplicate active requests.

Recommended indexes

`users`
email (unique)

`user_roles`
role, is_active

`applications`
status, job_role_id, user_id, applied_at

`mentor_assignments`
mentor_id, application_id, status

`assessment_rounds`
mentor_assignment_id, scheduled_at

`assessment_attempts`
round_id, status

`assessment_events`
attempt_id, event_timestamp

`introduction_requests`
company_id, candidate_id, status

`notifications`
user_id, is_read, created_at

Foreign-key columns are not indexed automatically in PostgreSQL. Composite primary keys already cover the leading columns of user_roles, user_skills and company_members.

Workflow rules and data handling

Design decisions that the tables rely on

RULES

Key workflow rules

Scoring is an aid

Automated scoring is an initial aid. Mentor evaluation and the round decision remain separate fields.

Monitoring is a signal

Browser monitoring events are signals only, not proof of misconduct. Disclose monitoring and retention.

Search is a restricted view

Company search is a restricted view. Reveal candidate contact details only after explicit candidate consent.

Eligibility needs a human

Mentor eligibility is checked from the resume analysis (over 10 years estimate) and then confirmed by human admin review.

Security and data handling

Server-side authorization

Enforce role-based authorization on the server for every API and database operation.

Row-level security

Use PostgreSQL RLS where appropriate, especially for company-facing candidate search and candidate-owned records.

Private resumes

Keep resumes in private object storage and issue short-lived, authorised access links.

No contact before consent

Candidate contact data must not appear in company search responses before consent.

Disclosed tracking

Candidates must be told about assessment event monitoring and engagement tracking.

UTC timestamps

Store TIMESTAMPTZ in UTC and display in the user's local timezone.

Use transactions

Wrap approval, assignment, assessment submission and introduction consent in transactions.

Question snapshots

Preserve question versions or snapshots for historical assessment attempts.

Observations and trade-offs

Things worth confirming before writing migrations, in rough order of impact

NOTES

1. One assessment plan per assignment

If a mentor needs several separate assessments for the same assignment, keep assessments as a table.

2. Question versioning is still open

assessment_questions can be edited after publishing. Snapshot or version before the first migration; a snapshot now copies one answer_key column instead of two.

3. One analysis per resume

Re-analysis overwrites the parse columns on resumes. Keep a separate table if you want a history.

4. One shared profile

A person who is both mentor and candidate has one title and bio. Split profiles if they must differ.

5. Decision history

assessment_rounds holds one current decision; earlier decisions go to audit_logs. Decide before building the reschedule and hold flows.

6. Fixed roles

Adding a sixth role needs a migration, not an INSERT. permission_key values must be validated in code.

7. Signals are not proof

Resume experience, assessment_events and AI scores are aids; each keeps the raw signal and the human outcome in different columns.

8. Consolidations to confirm

parsed_data JSONB groups skills, employment history and education; notifications keep message but no title; created_at is dropped on user_roles and assessment_rounds.

Scope: derived from the platform database architecture document (32 tables, 1 view). Validate migrations, access policies, retention rules and the question-versioning strategy against the exact workflows before production.

RECRUITMENT & ASSESSMENT PLATFORM — CASE STUDY

Technical Architecture & Application Flow

A complete, end-to-end view of the platform — from user onboarding to hiring, with system structure, data flow, and database design.

Goal
Assess candidates, match the right talent, and streamline hiring.

Type
End-to-end recruitment & assessment platform

Platform Actors

Candidate
Create profile, add skills, take assessments, and apply for jobs.

Mentor
Review assessments, evaluate responses, and give scores/feedback.

Company
Post jobs, search candidates, review assessments, and hire.

Admin
Manage users, roles, assessments, and platform operations.

Application Flow (Funnel View)

1. Onboarding
Sign up / Log in (single account for all roles)

2. Profile Setup
Add personal details, education and experience

3. Assessment
Take assigned tests and get evaluated (Auto / Mentor)

4. Job Application
Apply for relevant job openings (with assessment results)

5. Selection
Shortlist, interview and hire

Onboarding
User registration and account setup

Preparation
Build a complete candidate profile

Assessment
Skill evaluation and scoring

Hiring
Application review, shortlisting and selection

Database Structure (Core Tables)

Table	ID Type	Purpose
users	UUID	User account (all roles)
profiles	BIGINT	User profile details
user_skills	BIGINT	Skills added by users
skills	BIGINT	Master skills catalogue
resumes	BIGINT	Uploaded resume files & metadata
assessments	BIGINT	Assessment tests (title, type, config)
attempts	BIGINT	User test attempts and scores
applications	BIGINT	Job applications by candidates
job_roles	BIGINT	Jobs posted by companies
evaluation_reviews	BIGINT	Mentor evaluation and feedback

Key Relationships

- users (UUID) → profiles (1:1)
- users (UUID) → resumes (1:N)
- users (UUID) → user_skills (1:N)
- skills (BIGINT) → user_skills (1:N)
- users (UUID) → applications (1:N)
- job_roles (BIGINT) → applications (1:N)
- assessments (BIGINT) → attempts (1:N)
- attempts (BIGINT) → evaluation_reviews (1:1)

System & Data Flow

Users (All Roles) → **Platform Modules** (Profile, Skills, Assessment, Jobs) → **Database** (Core Tables) → **Processing** (Assessment, Evaluation, Matching) → **Outcome** (Shortlist, Hire, Reports)

Outcome

- Clear, structured user journey from signup to hiring.
- Centralized and normalized data across all modules.
- Role-based access with a single user account.
- Scalable, maintainable and easy-to-extend architecture.

RECRUITMENT & ASSESSMENT PLATFORM

Application Flow — Case Study

The complete, end-to-end flow of the platform — from user signup to hiring, with roles, key actions, and data flow.

Goal
Build a single platform where candidates get assessed, companies find the right talent, mentors review and evaluate, and admins manage it all.

Platform Flow

1. **Sign Up** — One account for all roles
2. **Build Profile** — Add details, skills and resume
3. **Assessment** — Take or review assessments
4. **Application** — Apply for jobs using assessment results
5. **Selection** — Company reviews, shortlists and hires

Role-wise Flow (What each user does)

Candidate — Find opportunities, get assessed and apply
Sign Up / Login → Create Profile & Add Skills → Upload Resume → Take Assessments → Apply for Jobs → Track Application

Mentor — Review, evaluate and provide assessment feedback
Login → View Assignments → Review Attempts → Evaluate & Give Scores → Submit Feedback → Evaluation Completed

Company — Post jobs, find and hire talent
Login → Create Company Profile → Post Job Openings → Search & Filter Candidates → View Assessments & Applications → Shortlist / Hire

Admin — Manage platform, users and assessments
Login → Manage Users & Roles → Create Assessments → Assign Mentors → Monitor Applications → Manage Platform

Key Outcomes

- Single account for all user types.
- Structured assessment and evaluation flow.
- Skill and resume based job matching.
- Mentor-reviewed, score-driven hiring.
- Company sourcing with candidate consent.
- Centralized platform management.

Database Flow (How the data connects)

users (UUID) → profiles (BIGINT) — User details → resumes (BIGINT) — Resume files → user_skills (BIGINT) — User + Skills → applications (BIGINT) — Job applications → assessments (BIGINT) — Tests & questions → attempts (BIGINT) — User attempts → evaluations (BIGINT) — Mentor scores → job_roles (BIGINT) — Company jobs

ID Strategy

- **users.id → UUID** (one account for all roles)
- **All other ids → BIGINT** (auto-increment)

RECRUITMENT & ASSESSMENT PLATFORM — CASE STUDY

CASE STUDY

Resumes, skills and applications

A detailed case study of how a candidate builds their profile, adds skills, uploads a resume and applies for a job, with the data flow and database design.

Flow in focus
Candidate profile → Skills → Resume → Job application

Goal
Show the end-to-end flow, the data it creates, and how the tables connect.

Scenario

A final-year student, Aisha, creates her profile on the platform, adds her skills, uploads her resume, and applies for a Frontend Developer role at a company.

Actors

Candidate (Aisha)
Creates profile, manages skills and resume, and applies for a job.

Company (TechNova)
Posts a job and receives applications.

Technology & Data Rules

- users.id is UUID (one account for all roles).
- All other id fields are BIGINT (auto-increment).
- A candidate can have many skills (via user_skills).
- A candidate can have many resumes (via resumes).
- A candidate can apply to many jobs (via applications).

The Flow: From Profile to Application

1 — Create account & profile
Aisha signs up, creating a user account (UUID) and completes her profile (education, bio, etc.).

2 — Add skills
She adds her skills (e.g., React, TypeScript, UI/UX) from the shared skill catalogue.

3 — Upload resume
She uploads her resume (PDF), which is stored with metadata and parse result (e.g., extracted skills).

4 — Apply for a job
She finds a Frontend Developer role at TechNova and submits an application, selecting her latest resume.

5 — Application recorded
The system creates an application record linking Aisha, the job, and her resume, with status = 'applied'.

Database Architecture (for this flow)

resumes — 5 cols

PK	id	BIGINT
FK	user_id	UUID
	file_url	TEXT
	file_name	VARCHAR(255)
	is_primary	BOOLEAN
	+2 more columns	

applications — 10 cols

PK	id	BIGINT
FK	user_id	UUID
FK	job_role_id	BIGINT
FK	resume_id	BIGINT
	application_type	VARCHAR(50)
	status	VARCHAR(50)
	+4 more columns	

job_roles — 3 cols

PK	id	BIGINT
	title	VARCHAR(200)
	status	BOOLEAN

user_skills — 4 cols

PK	id	BIGINT
FK	user_id	UUID
FK	skill_id	BIGINT
	proficiency	VARCHAR(20)

skills — 2 cols

PK	id	BIGINT
	skill_name	VARCHAR(100)

role_skills — 2 cols

PK, FK	job_role_id	BIGINT
PK, FK	skill_id	BIGINT

Sample Data (after Aisha applies)

users

id (UUID)	email
a7f2e8b4-...-9c3d	aisha@email.com

skills

id (BIGINT)	skill_name
1	React
2	TypeScript
3	UI/UX

user_skills

id	user_id (UUID)	skill_id	proficiency
1	a7f2e8b4-...	1	Advanced

resumes

id	user_id (UUID)	file_name
1	a7f2e8b4-...	Aisha_CV.pdf

job_roles

id	title
10	Frontend Developer

applications

id	user_id (UUID)	job_role_id	resume_id	status
1	a7f2e8b4-...	10	1	applied

How the Data Connects

1	**users (UUID)**	One account for the candidate (Aisha).
2	**user_skills → skills**	Links the user to selected skills from the shared catalogue.
3	**resumes**	Stores the uploaded resume and metadata (e.g., file name, primary).
4	**job_roles**	The job posted by a company (e.g., Frontend Developer).
5	**applications**	The application record that links the user, job role, and resume.

Outcome

- ✓ A complete candidate profile with skills and resume.
- ✓ One job application recorded and linked to the resume and candidate.
- ✓ Company can view and manage the application (e.g., review, shortlist, reject).

Result & Value

- Clear, normalized data flow from profile to application.
- Reuses a shared skills catalogue for consistency.
- Supports multiple resumes and multiple applications.
- Keeps a single user account (UUID) across the platform.
- Enables companies to efficiently review and manage applications.